GRUPO 8 · REQUERIMIENTOS FUNCIONALES

Reservas y Turnos

Módulo: Sistema de reserva de citas

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 8
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aiquaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre la agenda de turnos: la creación de una reserva para un servicio en una fecha y hora determinadas, su consulta y las transiciones de confirmación y cancelación.

Fuera de alcance: Disponibilidad de agenda, duración del turno, control de solapamientos, recursos o profesionales asignados, recordatorios y reprogramación: la reserva es un registro libre sobre una fecha declarada.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy `/api/proxy/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v1/reservas	RF-G8-01	Lista reservas activas, opcionalmente filtradas por titular.
POST /api/v1/reservas	RF-G8-02	Crea una reserva en estado pendiente.
GET /api/v1/reservas/{id}	RF-G8-05	Devuelve el detalle de una reserva.
PUT /api/v1/reservas/{id}	RF-G8-06	Reemplaza servicio, fecha/hora y notas de una reserva.
DELETE /api/v1/reservas/{id}	RF-G8-07	Da de baja (soft-delete) una reserva.
PATCH /api/v1/reservas/{id}/confirmar	RF-G8-03	Confirma una reserva.
PATCH /api/v1/reservas/{id}/cancelar	RF-G8-04	Cancela una reserva.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key.".
- Cada API key admite un máximo de **200 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.

- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` de la API hace baja lógica (`activo = false`), devolviendo `204 No Content` sin cuerpo.
- Casi todas las tablas de negocio (`sesiones`, `transferencias`, `facturas`, `tarjetas`, `notificaciones`, `ordenes`, `reservas`, `movimientos`, `roles`, además de `usuarios` y `cuentas` que ya la tenían) tienen columna `activo`/`activa`; todo `GET`, `PUT` y `DELETE` de esa tabla filtra u opera solo sobre filas con `activo = true` — una fila dada de baja deja de ser visible y de poder reemplazarse, pero no se borra.
- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- La creación requiere un titular existente; el `usuarioId` se obtiene del alta de cliente (RF-G4-01) o de los datos sembrados.
- La fecha y hora conviene enviarla en formato ISO 8601 con zona horaria (por ejemplo `2026-09-01T14:30:00-03:00`), que es el formato que produce la pantalla web.

3. Modelo de datos del módulo

reservas

Turnos agendados por un titular. Es la única tabla que toca el módulo.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`servicio`	`text`, obligatorio; texto libre
`fecha_hora`	`timestamptz`, obligatorio
`estado`	`text`, uno de `pendiente` / `confirmada` / `cancelada` / `completada`; por defecto `pendiente`
`notas`	`text`, opcional
`created_at`	`timestamptz`, por defecto `now()`
`activo`	`boolean`, obligatorio, por defecto `true`; lo usan `GET/PUT/DELETE` para filtrar/operar

> El estado `completada` existe en el modelo y aparece en los datos sembrados, pero ningún endpoint lo produce: no hay operación de cierre del turno.

4. Requerimientos funcionales

RF-G8-01 — Listar reservas

`GET /api/v1/reservas`

Devuelve las reservas del sandbox, con la posibilidad de acotar el resultado a un titular determinado. Es la agenda sobre la que operan la confirmación y la cancelación.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.

Reglas de negocio

- Devuelve únicamente reservas con `activo = true`; una dada de baja con RF-G8-07 deja de aparecer.
- Con `usuarioId`, devuelve todas las reservas de ese titular ordenadas por identificador ascendente.
- Sin `usuarioId`, devuelve las primeras 100 reservas del sandbox.
- El listado incluye reservas en cualquier `estado` de negocio, incluidas las canceladas.
- No hay filtro por rango de fechas ni ordenamiento por fecha del turno: el orden es por identificador.

Respuesta esperada

- `200 OK` con `data` como arreglo de reservas, cada una con `id`, `usuario_id`, `servicio`, `fecha_hora`, `estado`, `notas` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` presente pero no numérico, cero o negativo.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/reservas/route.ts`

RF-G8-02 — Crear una reserva

`POST /api/v1/reservas`

Agenda un turno para un titular, sobre un servicio y una fecha y hora determinados. El turno queda pendiente de confirmación.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del titular.
- `servicio` (obligatorio): texto no vacío; no está restringido a una lista de servicios.
- `fechaHora` (obligatorio): fecha y hora en formato ISO 8601 con zona horaria. La validación de entrada también admite cualquier texto no vacío, en cuyo caso la base de datos es la que decide si puede interpretarlo.
- `notas` (opcional): texto libre; si se omite se guarda vacío.

Reglas de negocio

- La reserva se crea siempre con `estado = 'pendiente'`; el cliente no puede elegir el estado inicial.
- **No se valida que la fecha sea futura:** se admite agendar un turno en el pasado.
- **No se valida disponibilidad ni solapamiento:** dos reservas del mismo servicio, a la misma hora y para el mismo titular son ambas aceptadas.
- El texto de fecha y hora que la base no logra interpretar produce un error de ejecución, no un error de validación.
- El titular debe existir: un `usuarioId` inexistente falla como error de ejecución por violación de clave foránea.

Respuesta esperada

- `201 Created` con `data` conteniendo la reserva creada.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, `servicio`/`fechaHora` están vacíos, o el titular no existe.
`EXECUTION_ERROR`	400	La fecha y hora enviadas no son interpretables como fecha.

Fuente: `app/api/v1/reservas/route.ts`

RF-G8-05 — Consultar una reserva

`GET /api/v1/reservas/{id}`

Devuelve el detalle de una reserva puntual.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve la reserva cuyo identificador coincide exactamente, siempre que siga `activo = true`.
- Si no existe o ya está dada de baja, responde 404 con el mensaje "Reserva no encontrada.".

Respuesta esperada

- `200 OK` con `data` conteniendo la reserva completa.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una reserva vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/reservas/[id]/route.ts`

RF-G8-06 — Reemplazar una reserva

`PUT /api/v1/reservas/{id}`

Reemplaza el servicio, la fecha/hora y las notas de una reserva existente. El estado queda fuera de este reemplazo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `servicio` (obligatorio): texto no vacío.
- `fechaHora` (obligatorio): fecha y hora en formato ISO 8601 con zona horaria, o cualquier texto no vacío.
- `notas` (opcional): texto libre.

Reglas de negocio

- Solo puede reemplazar una reserva con `activo = true`; una dada de baja responde 404.
- `estado` no forma parte del cuerpo: confirmar/cancelar sigue siendo exclusivo de RF-G8-03/04.
- Igual que en la creación, no se valida que la nueva fecha sea futura ni que no se solape con otra reserva.

Respuesta esperada

- `200 OK` con `data` conteniendo la reserva ya actualizada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una reserva vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta un campo obligatorio.
`EXECUTION_ERROR`	400	La nueva `fechaHora` no es interpretable como fecha.

Fuente: `app/api/v1/reservas/[id]/route.ts`

RF-G8-07 — Dar de baja una reserva

`DELETE /api/v1/reservas/{id}`

Da de baja lógica una reserva, quitándola de los listados y consultas por id.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false`; la fila permanece en la base, nunca se borra físicamente.
- Una reserva en cualquier `estado` de negocio (pendiente, confirmada, cancelada) puede darse de baja.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una reserva vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/reservas/[id]/route.ts`

RF-G8-03 — Confirmar una reserva

`PATCH /api/v1/reservas/{id}/confirmar`

Confirma un turno agendado, dejándolo en firme.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo de la reserva.

Reglas de negocio

- El estado de la reserva pasa a `confirmada` sin verificar el estado anterior.
- La operación es idempotente: confirmar una reserva ya confirmada devuelve el mismo resultado.
- **Una reserva cancelada puede confirmarse**, porque no hay máquina de estados que lo impida.
- No se registra quién confirma ni cuándo se confirmó, más allá de la bitácora general de requests.
- **No filtra por `activo`:** puede confirmar una reserva ya dada de baja con RF-G8-07, aunque esa reserva ya no aparezca en ningún listado.
- Si la reserva no existe, responde 404 y no modifica nada.

Respuesta esperada

- `200 OK` con `data` conteniendo la reserva ya confirmada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una reserva con ese identificador (mensaje: "Reserva no encontrada.").
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/reservas/[id]/confirmar/route.ts`

RF-G8-04 — Cancelar una reserva

`PATCH /api/v1/reservas/{id}/cancelar`

Cancela un turno agendado, liberando la intención de asistir.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo de la reserva.

Reglas de negocio

- El estado de la reserva pasa a `cancelada` sin verificar el estado anterior.
- La operación es idempotente: cancelar una reserva ya cancelada devuelve el mismo resultado.
- La cancelación no borra la reserva: la fila permanece y sigue apareciendo en los listados (mientras siga `activo = true`).
- No hay plazo mínimo de antelación ni penalidad por cancelar: una reserva puede cancelarse en cualquier momento, incluso pasada su fecha.
- **No filtra por `activo`:** puede cancelar una reserva ya dada de baja con RF-G8-07.
- Si la reserva no existe, responde 404 y no modifica nada.

Respuesta esperada

- `200 OK` con `data` conteniendo la reserva ya cancelada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una reserva con ese identificador (mensaje: "Reserva no encontrada.").
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/reservas/[id]/cancelar/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body.".
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`, ej. un email o número de factura repetido) se traduce a `409 CONFLICT`; una violación de clave foránea o de

restricción `CHECK` (`23503` / `23514`) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos cae en `400 EXECUTION_ERROR`.

- `DELETE` **es siempre baja lógica:** actualiza `activo = false` (o `activa` en `cuentas`) y responde `204 No Content` **sin cuerpo** — no `200` con la fila afectada.
- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera Next.js automáticamente, ninguna ruta lo arma a mano, y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear`, `activar`, `confirmar`, `cancelar`, `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.
- **Las transiciones de estado anteriores al CRUD no miran** `activo`**:** `PATCH .../bloquear`, `.../activar`, `.../leer`, `.../confirmar`, `.../cancelar`, `.../kyc` y `POST .../pagar` no filtran por `activo = true` (ninguno de esos archivos se tocó al agregar el CRUD). Un recurso dado de baja con el nuevo `DELETE` deja de aparecer en `GET`, pero **sigue pudiendo bloquearse, confirmarse, leerse o pagarse** a través de su acción dedicada — una inconsistencia real del sandbox, no un comportamiento a asumir como intencional.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso; también un POST de upsert que reactivó una fila existente en vez de crearla.
`201 Created`	POST que crea una fila nueva.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Falta el header `x-api-key`, o la clave no existe o está inactiva.
`RATE_LIMITED`	429	Se superaron las 200 requests por minuto de esa API key.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe, ya está dado de baja (`activo=false`), o no está en un estado que admita la operación.
`CONFLICT`	409	Violación de restricción `UNIQUE` en Postgres (ej. email, número de factura o nombre de rol duplicado).
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local. **La interfaz web todavía no tiene pantallas para `PUT` / `DELETE` ni para los recursos nuevos (`sesiones`, `movimientos`)**: solo cubre los flujos de creación y consulta que ya existían; esos requerimientos se prueban llamando la API directamente (Postman, `curl`, o el cliente HTTP del framework de automatización), no navegando la web.

Pantalla	Qué permite hacer
`/reservas` — Agenda	Tabla con servicio, fecha y hora, notas y estado, con filtro por titular y botones de confirmar y cancelar por fila. Ejecuta RF-G8-01, RF-G8-03 y RF-G8-04.
`/reservas/new` — Nueva reserva	Formulario con titular, servicio, fecha y hora y notas opcionales. Ejecuta RF-G8-02 y, al crearse, vuelve a la agenda.

- El campo de fecha y hora del formulario es un selector del navegador, que envía un valor con formato válido: los escenarios de fecha inválida requieren llamar la API directamente.
- Las acciones de confirmar y cancelar refrescan la agenda, de modo que el nuevo estado se ve sin recargar la página.
- No hay pantalla de detalle, edición ni baja individual: `GET`, `PUT` y `DELETE /reservas/{id}` (RF-G8-05/06/07) solo se pueden ejercitar llamando la API directamente.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G8-01 — Listar reservas

- Al listar reservas filtrando por un titular, todos los elementos devueltos tienen ese `usuario_id`.
- El listado incluye reservas canceladas junto a las pendientes y confirmadas.
- Al filtrar por un titular sin reservas, la respuesta es 200 con `data` vacío.
- Una reserva recién creada aparece en el listado de su titular.

RF-G8-02 — Crear una reserva

- Al crear una reserva con datos válidos, la respuesta es 201 y `data.estado` es `pendiente`.
- La fecha y hora devueltas corresponden al instante enviado, expresadas con zona horaria.
- Al crear una reserva sin `notas`, la operación es aceptada y el campo queda vacío.
- Al crear una reserva con una fecha ya pasada, la respuesta es 201: el sandbox no valida la vigencia.
- Al crear dos reservas con el mismo servicio, titular y horario, ambas son aceptadas: no hay control de solapamiento.
- Al enviar `fechaHora` con un texto no interpretable como fecha, la respuesta es 400 `EXECUTION_ERROR`.
- Al crear una reserva para un `usuarioId` inexistente, la respuesta es 400 `VALIDATION_ERROR`.

RF-G8-05 — Consultar una reserva

- Dada una reserva existente, al consultarla por su `id` la respuesta es 200 y los datos coinciden con los de la creación.
- Al consultar una reserva inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.

RF-G8-06 — Reemplazar una reserva

- Dada una reserva existente, al reemplazar su `servicio` la respuesta es 200 y el cambio se refleja al consultarla.
- El `estado` de la reserva no cambia por este reemplazo.
- Al reemplazar con una `fechaHora` no interpretable, la respuesta es 400 `EXECUTION_ERROR`.
- Al reemplazar una reserva inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.

RF-G8-07 — Dar de baja una reserva

- Dada una reserva vigente, al darla de baja la respuesta es 204 sin cuerpo.
- Tras la baja, `GET /reservas/{id}` sobre esa reserva responde 404, y deja de aparecer en RF-G8-01.

RF-G8-03 — Confirmar una reserva

- Dada una reserva pendiente, al confirmarla la respuesta es 200 y `data.estado` es `confirmada`.
- Al confirmar dos veces la misma reserva, ambas respuestas son 200 y el estado final es `confirmada`.
- Al confirmar una reserva previamente cancelada, la respuesta es 200 y la reserva vuelve a estar confirmada.
- Al confirmar una reserva inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Reserva no encontrada.".

RF-G8-04 — Cancelar una reserva

- Dada una reserva confirmada, al cancelarla la respuesta es 200 y `data.estado` es `cancelada`.
- Tras la cancelación, la reserva sigue apareciendo en el listado del titular con estado `cancelada`.

- Al cancelar dos veces la misma reserva, ambas respuestas son 200.
- Al cancelar una reserva cuya fecha ya pasó, la respuesta es 200: no hay plazo que lo impida.
- Al cancelar una reserva inexistente, la respuesta es 404 `NOT_FOUND`.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Ley 1334/98 de Defensa del Consumidor (SEDECO), constancia de la contratación	El consumidor debe recibir constancia del turno reservado con servicio, fecha y hora.	Cubierto: la respuesta devuelve la reserva completa y la agenda muestra el estado actualizado.
Ley 1334/98, condiciones de cancelación	Las condiciones, plazos y penalidades de cancelación deben informarse previamente y aplicarse de forma clara.	No implementado: la cancelación no tiene plazo ni penalidad, y las condiciones no forman parte del modelo.
Ley 4868/2013 de Comercio Electrónico, confirmación de la operación	La confirmación del turno debe ser un acto verificable y trazable.	Parcial: existe la operación de confirmación y queda auditada como request, pero no se registra quién confirmó ni cuándo dentro de la reserva.
Buenas prácticas de agenda de servicios	El sistema debe impedir turnos superpuestos y turnos en horarios no disponibles.	No implementado: no hay disponibilidad, duración ni control de solapamiento.
Ley 6534/2020 de protección de datos personales	Los datos de la agenda de un titular solo deben ser accesibles para ese titular.	No implementado: el listado sin filtro devuelve reservas de todos los titulares y cualquiera puede confirmarlas o cancelarlas.

Brechas conocidas del sandbox

- Las transiciones de estado no validan el estado previo: una reserva cancelada puede volver a confirmarse.
- El estado `completada` nunca se produce por API.
- No se valida que la fecha del turno sea futura ni que haya disponibilidad; no existe control de solapamiento.
- El campo `servicio` es texto libre, sin catálogo de servicios ofrecidos.
- No hay recordatorios ni notificaciones asociadas a la reserva, pese a que el sandbox tiene un módulo de notificaciones.
- El listado no admite filtro por fecha ni por estado, y está limitado a 100 registros sin paginación.
- `PATCH .../confirmar` y `.../cancelar` no filtran por `activo`: pueden operar sobre una reserva ya dada de baja con `DELETE`.